Exercicis amb LED, PIR, Sensor de Distància i Buzzer a Raspberry Pi amb Python

Exercici 1: Control d'un LED amb un sensor PIR usant interrupcions

Enunciat: Configura un sistema on un LED s'encengui cada vegada que es detecti moviment amb un sensor PIR. Quan el PIR detecti moviment, el LED s'activarà automàticament durant uns segons i després s'apagarà. Utilitza una interrupció configurada per una pujada de senyal que permeti controlar l'activació del LED en funció del senyal del PIR.

```
import RPi.GPIO as GPIO
import time

PIR_PIN = 23
LED_PIN = 17

GPIO.setmode(GPIO.BCM)
GPIO.setup(PIR_PIN, GPIO.IN)
GPIO.setup(LED_PIN, GPIO.OUT)

def encendre_led(channel):
   GPIO.output(LED_PIN, True)
   time.sleep(2)  # LED encès per 2 segons
   GPIO.output(LED_PIN, False)

GPIO.add_event_detect(PIR_PIN, GPIO.RISING, callback=encendre_led)

try:
   while True:
      time.sleep(1)  # Manté el programa actiu
except KeyboardInterrupt:
   print("Programa finalitzat")
finally:
   GPIO.cleanup()
```

Exercici 2: Activació d'un buzzer segons distància amb interrupcions

Enunciat: Crea un sistema d'alerta amb un sensor de distància i un buzzer. Quan la distància sigui menor de 15 cm, el buzzer ha de sonar, i quan la distància superi aquest valor, s'ha de silenciar. Utilitza una interrupció que s'activa quan hi ha un canvi de distància per controlar l'activació del buzzer de manera eficient i només quan cal.

```
import RPi.GPIO as GPIO
import time

TRIG = 23
ECHO = 24
BUZZER_PIN = 18

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

def mesurar_distancia():
  GPIO.output(TRIG, True)
  time.sleep(0.00001)
  GPIO.output(TRIG, False)

  while GPIO.input(ECHO) == 0:
    start_time = time.time()
  while GPIO.input(ECHO) == 1:
    end_time = time.time()

  duration = end_time - start_time
  distance = (duration * 34300) / 2
  return distance

def activar_buzzer(channel):
  distancia = mesurar_distancia()
  if distancia < 15:
    GPIO.output(BUZZER_PIN, True)
  else:
```

```
        GPIO.output(BUZZER_PIN, False)

GPIO.add_event_detect(ECHO, GPIO.BOTH, callback=activar_buzzer)

try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    print("Programa finalitzat")
finally:
    GPIO.cleanup()
```

Exercici 3: Control de dos LEDs amb intervals de distància

Enunciat: Dissenya un sistema amb dos LEDs que reaccionen a diferents intervals de distància. El primer LED s'encendrà quan la distància sigui inferior a 10 cm, indicant proximitat extrema, mentre que el segon LED s'encendrà quan la distància estigui entre 10 cm i 20 cm, indicant una proximitat moderada. Utilitza interrupcions per activar i desactivar els LEDs segons aquests intervals de distància.

```
import RPi.GPIO as GPIO
import time

TRIG = 23
ECHO = 24
RED_LED_PIN = 17
YELLOW_LED_PIN = 27

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.setup(RED_LED_PIN, GPIO.OUT)
GPIO.setup(YELLOW_LED_PIN, GPIO.OUT)

def mesurar_distancia():
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
```

```
    GPIO.output(TRIG, False)

    while GPIO.input(ECHO) == 0:
        start_time = time.time()
    while GPIO.input(ECHO) == 1:
        end_time = time.time()

    duration = end_time - start_time
    distance = (duration * 34300) / 2
    return distance

def controlar_leds(channel):
    distancia = mesurar_distancia()
    if distancia < 10:
        GPIO.output(RED_LED_PIN, True)
        GPIO.output(YELLOW_LED_PIN, False)
    elif 10 <= distancia <= 20:
        GPIO.output(RED_LED_PIN, False)
        GPIO.output(YELLOW_LED_PIN, True)
    else:
        GPIO.output(RED_LED_PIN, False)
        GPIO.output(YELLOW_LED_PIN, False)

GPIO.add_event_detect(ECHO, GPIO.BOTH, callback=controlar_leds)

try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    print("Programa finalitzat")
finally:
    GPIO.cleanup()
```

Exercici 4: Alarmes seqüencials amb PIR i buzzer

Enunciat: Crea un sistema on, cada vegada que el sensor PIR detecti moviment, s'activi un buzzer per 1 segon i un LED per 2 segons en seqüència. Això actua com una alarma seqüencial on el buzzer s'activa primer per avisar del moviment detectat i, tot seguit, el LED s'encén per mantenir la indicació visual durant més temps. Utilitza interrupcions per detectar el moviment i activar els dispositius de manera seqüencial.

```
import RPi.GPIO as GPIO
import time

PIR_PIN = 23
BUZZER_PIN = 18
LED_PIN = 17

GPIO.setmode(GPIO.BCM)
GPIO.setup(PIR_PIN, GPIO.IN)
GPIO.setup(BUZZER_PIN, GPIO.OUT)
GPIO.setup(LED_PIN, GPIO.OUT)

def alarma_seqüencial(channel):
   GPIO.output(BUZZER_PIN, True)
   time.sleep(1)
   GPIO.output(BUZZER_PIN, False)

   GPIO.output(LED_PIN, True)
   time.sleep(2)
   GPIO.output(LED_PIN, False)

GPIO.add_event_detect(PIR_PIN, GPIO.RISING, callback=alarma_seqüencial)

try:
   while True:
      time.sleep(1)
except KeyboardInterrupt:
   print("Programa finalitzat")
finally:
```

```
    GPIO.cleanup()
```

Exercici 5: Alerta amb Buzzer i LED segons distància amb dues interrupcions

Enunciat: Dissenya un sistema que faci sonar un buzzer i encengui un LED verd quan la distància sigui inferior a 10 cm, indicant proximitat extrema. Quan la distància sigui superior a 20 cm, el buzzer ha d'apagar-se i s'ha d'encendre un LED blau, indicant que l'objecte es troba a una distància segura. Utilitza interrupcions per detectar canvis en la distància i per activar o desactivar els LEDs i el buzzer segons els intervals establerts.

```
import RPi.GPIO as GPIO
import time

TRIG = 23
ECHO = 24
BUZZER_PIN = 18
GREEN_LED_PIN = 17
BLUE_LED_PIN = 27

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.setup(BUZZER_PIN, GPIO.OUT)
GPIO.setup(GREEN_LED_PIN, GPIO.OUT)
GPIO.setup(BLUE_LED_PIN, GPIO.OUT)

def mesurar_distancia():
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    while GPIO.input(ECHO) == 0:
        start_time = time.time()
    while GPIO.input(ECHO) == 1:
        end_time = time.time()

    duration = end_time - start_time
```

```
    distance = (duration * 34300) / 2
    return distance

def alerta_buzzer_leds(channel):
    distancia = mesurar_distancia()
    if distancia < 10:
        GPIO.output(BUZZER_PIN, True)
        GPIO.output(GREEN_LED_PIN, True)
        GPIO.output(BLUE_LED_PIN, False)
    elif distancia > 20:
        GPIO.output(BUZZER_PIN, False)
        GPIO.output(GREEN_LED_PIN, False)
        GPIO.output(BLUE_LED_PIN, True)
    else:
        GPIO.output(BUZZER_PIN, False)
        GPIO.output(GREEN_LED_PIN, False)
        GPIO.output(BLUE_LED_PIN, False)

GPIO.add_event_detect(ECHO, GPIO.BOTH, callback=alerta_buzzer_leds)

try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    print("Programa finalitzat")
finally:
    GPIO.cleanup()
```